ee2004

Week7, 8

Bit Values as Voltage Levels

- If we start interacting the microcontroller with the outside world, then abstract representations such as logic level 0 or 1 do not mean much. Therefore, we should associate these logic levels with actual voltage values used in the microcontroller.
- There are several voltage standards to represent logic levels 0 and 1 in digital systems. The STM32F4 microcontroller supports 3.3 V standard low voltage complementary metal-oxide semiconductor (LVCMOS) and transfer logic (LVTTL) levels.
- In this figure, *VDD* represents the supply voltage of the microcontroller.

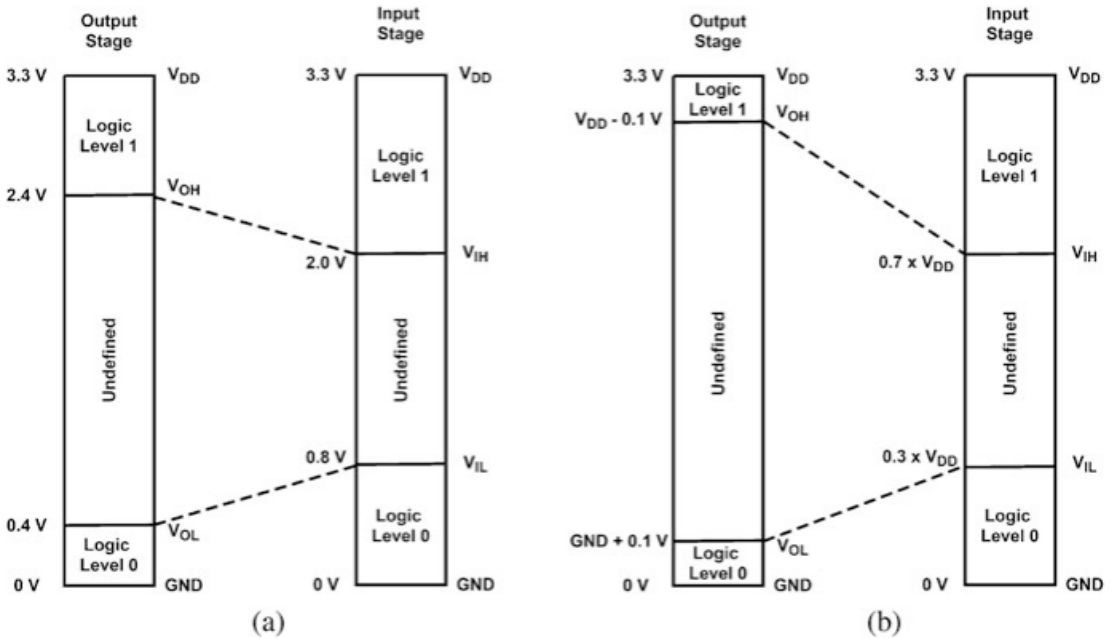


Fig. 4.1 3.3 V logic levels. (**a**) LVTTL. (**b**) LVCMOS

- When logic level 1 is fed to output from a pin, the voltage value there will be between *VDD* and *VOH* for both LVTTL and LVCMOS.
- Likewise, when logic level 0 is fed to output, the voltage value at the pin will be between 0 V and *VOL*.
- The voltage value to logic level correspondence also applies to input pin of the microcontroller.
- When we apply a voltage value between *VDD* and *VIH* to a pin, it will be recognized as logic level 1 in input.
- Likewise, applying a voltage value between *VIL* and 0 V to an input pin will be recognized as logic level 0.
- Hence, there are tolerance values for both input and output voltage values.
- This should be taken into account during digital I/O operations.
- Voltage values outside the tolerances are taken as uncertain within the microcontroller.

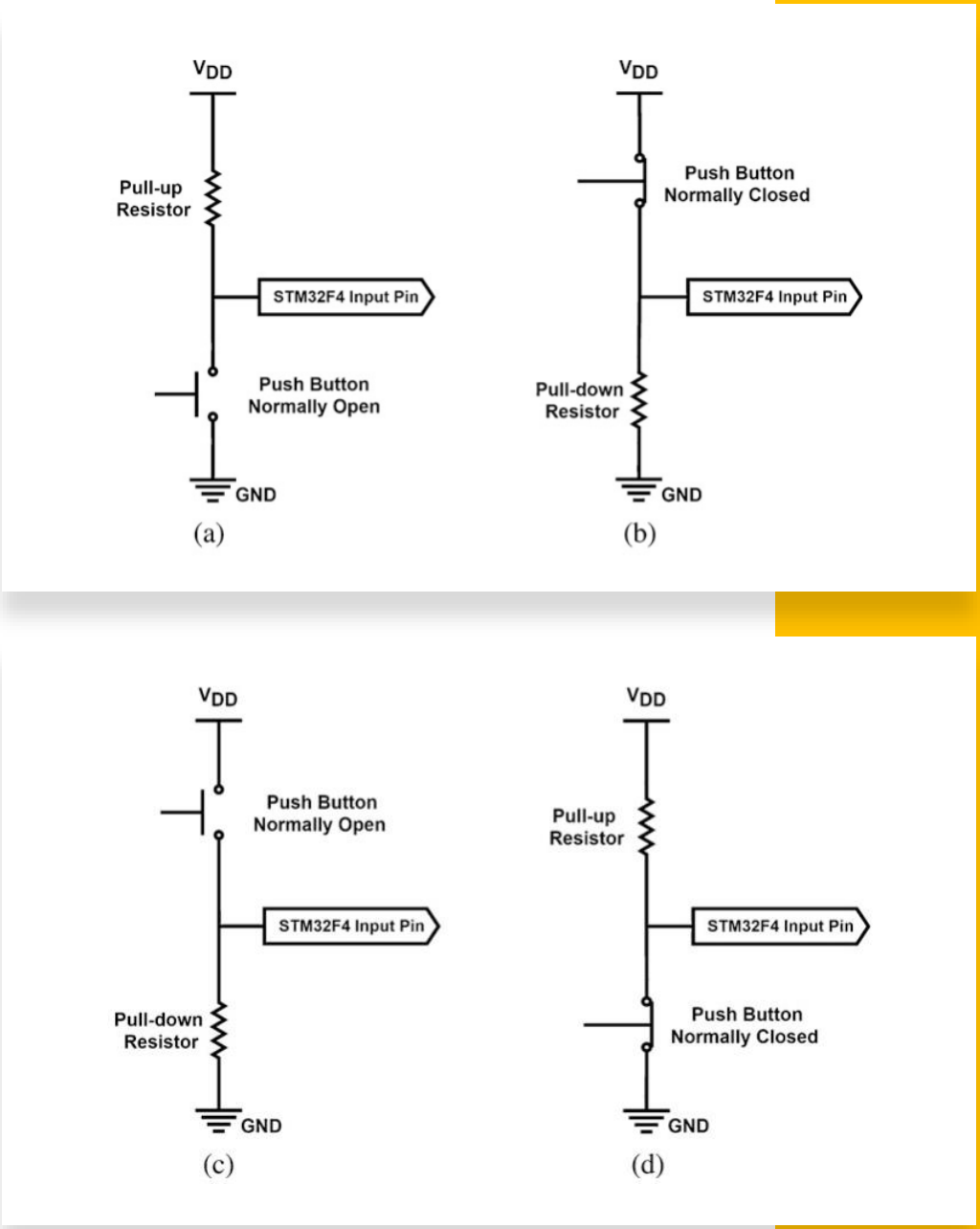
Digital Input from a Switch or Button

- While using a GPIO pin of the microcontroller, some applications may require special consideration. These can originate from input with contact closures (such as switch or button) or input with voltage values outside the microcontroller's tolerance levels. Likewise, when the microcontroller drives output loads with high current requirements, the user should be cautious.
- We can get digital input from a switch or button. In this setup, when the button is pressed (or switch is closed), a voltage value is applied to the microcontroller pin.
- The user should pay attention to two issues here. The first one is the setup (and related hardware) required for interfacing the switch or button to the microcontroller. The second one is avoiding switch bouncing.

Setting Up the Switch or Button

- There are two input setup options when a switch or button is connected to the pin of a microcontroller. In the first setup, the microcontroller gets *VDD* V (logic level 1) on its pin when the button (or switch) is interacted. Likewise, it gets 0 V (logic level 0) when the button or switch is left to its natural state. This is called active high input.
- In the second setup, the microcontroller gets 0 V (logic level 0) on its pin when the button (or switch) is interacted. It gets *VDD* V (logic level 1) when the button or switch is left to its natural state. This is called active low input.
- Active high or low inputs are not related to the microcontroller. Depending on the application, the input can be adjusted as active high or low using a pull-up or pull-down resistor. If input pin is left floating, then the input state cannot be determined.

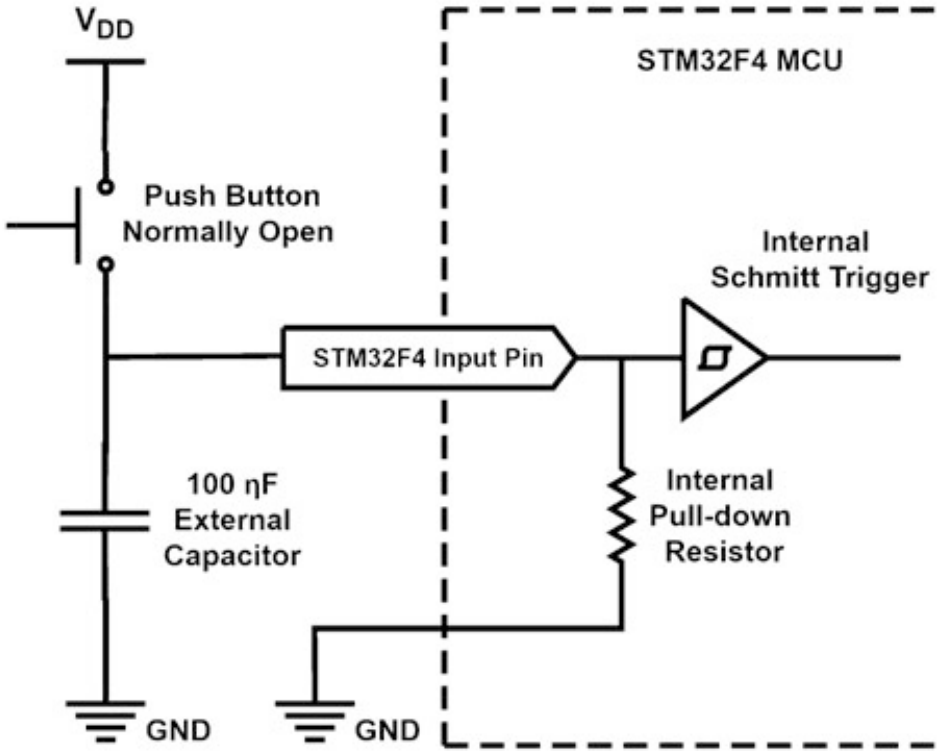
- The pull-up and pulldown resistors are used to pull the input to *VDD* or 0 V when the normally closed or open switch or button is interacted.
- These resistors can be externally connected or internal pull-up/pull-down resistors of the microcontroller can be used. Sample circuit diagrams for the active high/low input setup using normally opened and closed push buttons are as in Fig. 4.2.
- Active high and low input circuit diagrams for the push button. (**a**) Active low input, normally open push button. (**b**) Active low input, normally closed push button. (**c**) Active high input, normally open push button. (**d**) Active high input, normally closed push button



Avoiding Switch Bouncing

- Switch bouncing is a serious problem when a button (or switch) is used in a digital system.
- Bouncing may occur due to the following reason. If the button (or switch) is interacted or activated, two metal parts connect and disconnect several times before an actual stable connection. Hence, the device may generate output more than once, typically 10 to 100 times over a period of about 1 ms.
- GPIO pins of the microcontroller can capture these input values which result in false or multiple triggering while reading the input in the code.
- Either a software- or hardware-based solution can be used to eliminate the switch bouncing problem.
- The simplest software solution is adding a delay while reading the input in the code. Hence, once an input is captured, successive inputs are ignored during the delay. Although this method is easy to implement, actual inputs will also be eliminated at the same time. Moreover, this solution may block other operations and can cause unpredictable problems in the code. Therefore, it should be used with precaution.

- We can also use a shift register-based software solution to avoid switch bouncing.
- In this setup, the microcontroller captures the input values in 1 ms or a smaller time interval. These values are shifted through a shift register. During switch bouncing, the shift register content will be formed by logic level 0 and 1 values. However, the content will be formed by the same value after the bouncing effect. When such a pattern is observed, the stable input can be captured. This method may also block other software routines. However, the blocking period is usually much shorter compared to the plain delay method.
- There are also hardware-based solutions to avoid the switch bouncing problem. To do so, the most feasible circuitry is adding a low-pass RC filter (composed of a resistor and capacitor) followed by Schmitt trigger to the pin.



- Each digital I/O pin of the STM32F4 microcontroller has a Schmitt trigger.
- The internal pull-down resistor available at the pin hardware can be used in forming the RC filter.
- Resistance values for these are typically 40 K.
- An external capacitor should be connected to the pin as well. Hence, the actual setup to eliminate switch bouncing via RC circuitry will be as in Fig. 4.3 based on the available hardware.

Digital I/O with High Voltage Values

- Digital input to the microcontroller can originate from a source, such as sensor or external device, with high voltage values corresponding to logic level 0 and 1.
- Directly connecting such a source to the microcontroller can damage or burn out the GPIO pin circuitry. It may even damage other parts of the microcontroller.
- Likewise, some loads may require high voltage values to operate. Hence, the microcontroller should feed the necessary voltage levels to them.
- There are two methods to handle such situations. The first one is using a level shifter between the high voltage source or load and microcontroller. This method can be used for both high voltage input and output operations.
- The second method is using high voltage tolerant pins of the microcontroller, if there are any. To note here, even if there are such pins on the microcontroller, they can only be used for input.

- Let’s first consider level shifters with two options as unidirectional and bidirectional setup.
- Within the unidirectional setup, we can only convert a high voltage to a low value. In the bidirectional setup, we can convert both high and low voltage values to each other.
- The basic unidirectional high to low voltage level shifter is in fact a voltage divider formed by resistors. This shifter contains two resistors, as R1 and R2, which divide the high voltage by the ratio R1/(R1 + R2).

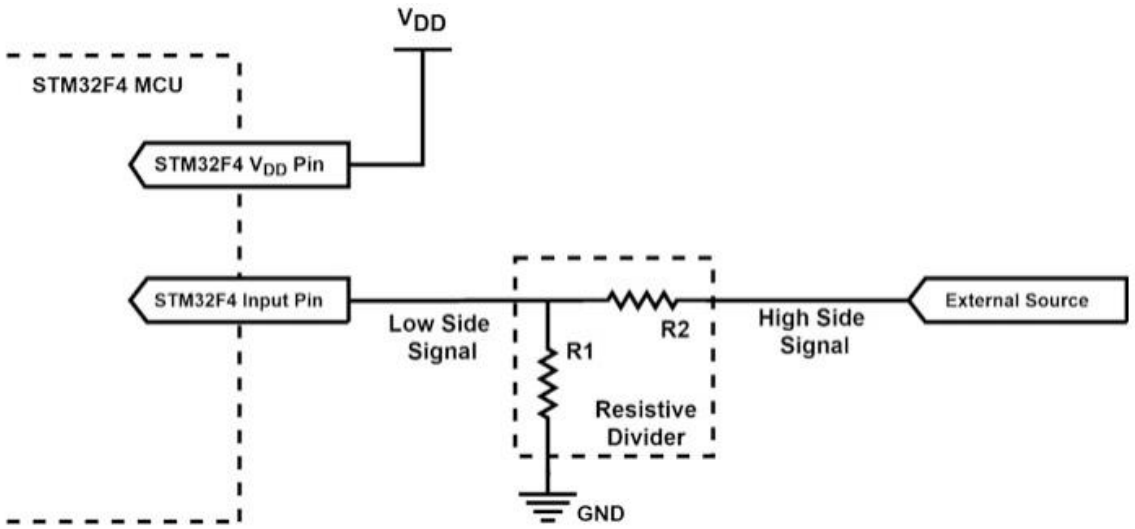


Fig. 4.4 Basic unidirectional voltage level shifter

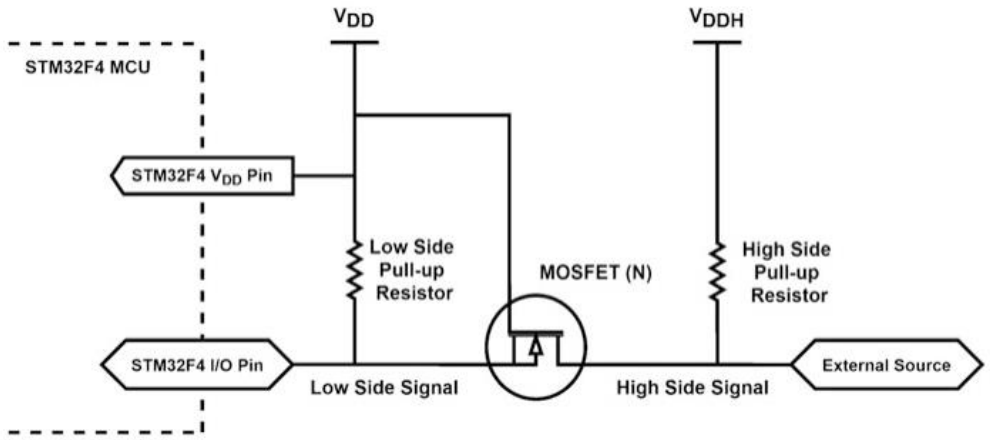


Fig. 4.5 Basic bidirectional voltage level shifter

- The basic bidirectional voltage level shifter contains a single N-channel MOSFET and a pair of pull-up resistors.
- We can divide the circuit in this figure into two parts as low and high side.
- Let's consider the scenario in which the low side is set as output and high side as input. Here, when the low side signal is set to V*DD*, the MOSFET turns off, and high side signal is pulled up to *VDDH* (high voltage level) through the pull-up resistor.
- When the low side signal is set to 0 V, the MOSFET source pin is set to ground voltage. Hence, the MOSFET turns on and the high side is pulled down to 0 V.
- Let's next consider the scenario in which the high side is set as output and low side as input. Here, the MOSFET substrate diode conducts current and pulls the low side signal down to 0.7 V when the high side signal is set to 0 V. When the high side signal is set to *VDDH*, the substrate diode does not conduct current. Hence, the low side signal is pulled up to *VDD* through the pull-up resistor.

Digital Output to a Load Requiring High Current and Voltage Values

- Most microcontrollers can feed 3.3–5 V and 25–40 mA through their GPIO pins. These values are 3.3 V and 25 mA for separate pins of the STM32F4 microcontroller.
- Besides, the total output current that can be drawn from all pins of the STM32F4 microcontroller at once cannot exceed 120 mA.
- We may want to drive heavy loads, such as a motor or power LEDs, by the microcontroller. Voltage and current output from the microcontroller may not be sufficient for them.
- There are three methods to handle these cases using transistors, optocouplers, and relays.
- The first method in handling a load requiring high voltage and current is using a transistor circuit between the load and microcontroller pin. This setup is only suitable for DC voltages. The reader can use either a BJT or MOSFET for this purpose.

- As can be seen in Fig. 4.6, BJT or MOSFET turns off when the microcontroller pin is set to logic level 0 (in other words 0 V). Hence, the current cannot flow through the load.
- When the microcontroller pin is set to logic level 1 (in other words 3.3 V), small current flows through the bias resistor and base-emitter junction of the BJT. Hence, BJT turns on and current flows from the collector (connected to load) to the emitter pin (connected to ground).
- MOSFET turns on and current flows from drain to source (connected to ground). To note here, *VDD*2 is the separate DC power supply voltage where load is connected to. The *VDD*2 voltage level can be same or higher than *VDD*.

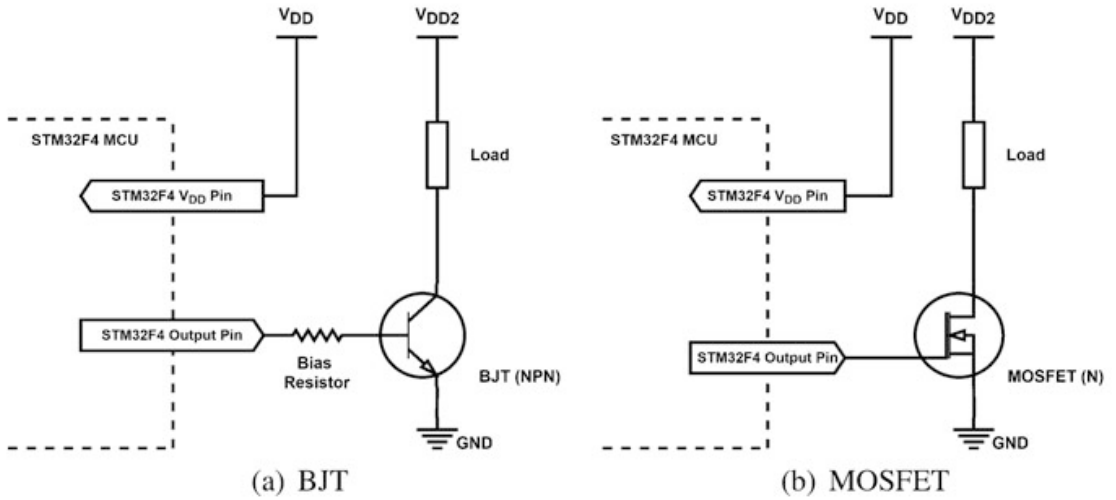
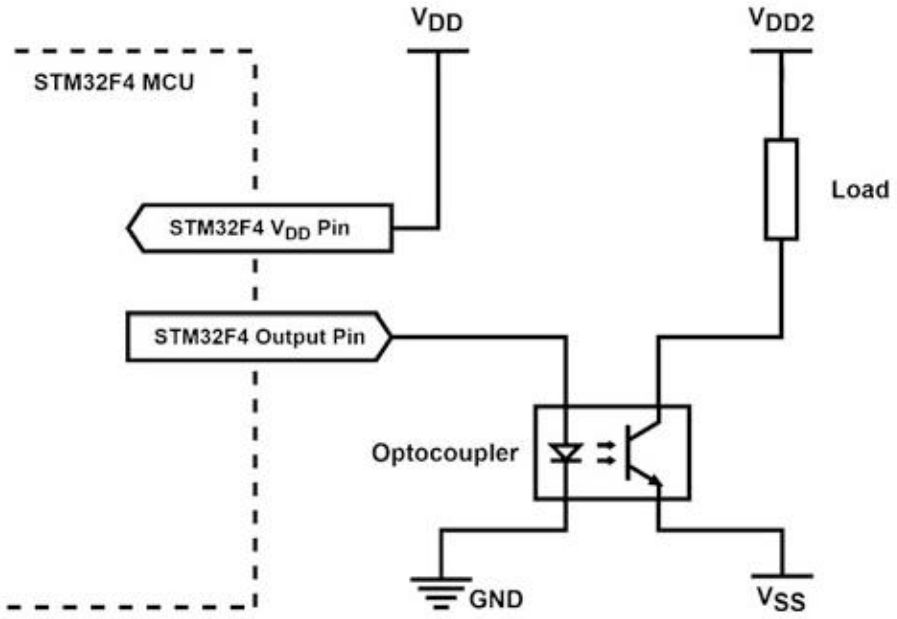


Fig. 4.6 Driving a heavy load with transistor. (**a**) BJT. (**b**) MOSFET

- The second method in handling a load requiring high voltage and current is using an optocoupler between the load and microcontroller pin. This setup is only suitable for DC voltages. Optocoupler is an electronic component made up by a simple phototransistor and LED that separates two circuits from each other using light. This setup allows handling loads with current and voltage values up to 3 A and 20 V, respectively. Moreover, the optocoupler completely separates two sides of the circuit (load and microcontroller) including their ground. This means different supply voltages and grounds can be used in the microcontroller and load connected to it.



- The third method in handling a load requiring high voltage and current is using a relay between the load and microcontroller pin.
- Unlike the previous two options, a relay can control both AC and DC voltages.
- Relay is an electromechanical or solidstate element that can open and close. The electromechanical relay is composed of a contact switch that opens or closes when current flows in its internal coil or coils.
- It can handle very high loads having current and voltage values such as 10– 15 A or 110–220 V, respectively.
- The solid-state relay (SSR) usually consists of an optocoupler at its input stage and triac or MOSFET at its output stage for AC or DC output, respectively.
- SSRs can handle very high loads such as the ones requiring 500 A current.
- Some small power, electromechanical or solid state, relays can be directly connected to the GPIO pin of the microcontroller. However, a transistor or another supplementary circuit is required to drive the relay itself most of the times.

- As can be seen in Fig. 4.8, when the microcontroller pin is set to logic level 0 (in other words 0 V), BJT turns off and current cannot flow through internal coil of the relay. Hence, internal contact switch does not close and load current does not flow.
- When the microcontroller pin is set to logic level 1 (in other words 3.3 V), BJT turns on and current flows through internal coil of the relay. This closes the internal contact switch of relay and high current can flow through the load.
- The diode connected against the coil is of type fly back. It protects the circuit from any back electromotive force (EMF) by the coil of the relay.
- To note here, *VDD*2 is the separate DC power supply voltage where the internal coil of relay is connected to. The *VDD*2 voltage level can be same or higher than *VDD*.
- *VDD*3 is the separate AC or DC power supply voltage where the load is connected to. The *VDD*3 voltage level can be any value that the relay can support.

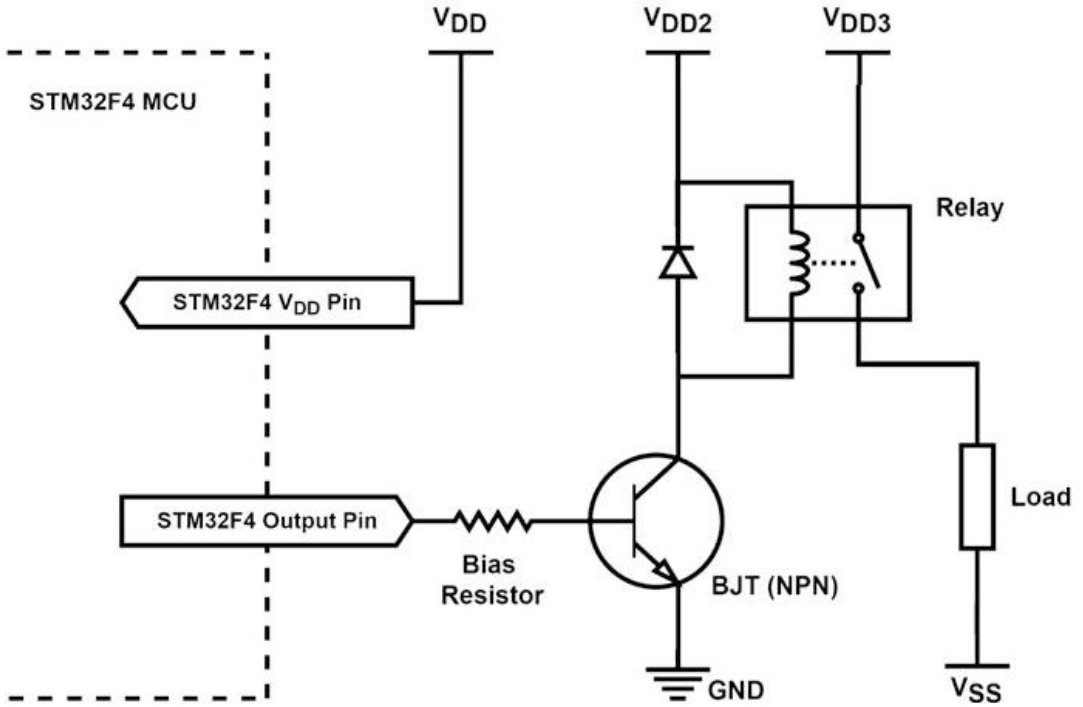


Fig. 4.8 Driving a heavy load with relay

Setting Up GPIO Registers

- STM32CubeIDE allows a simpler method to set up port registers when the C language is used in programming. To be more specific, HAL library functions are used for this purpose. HAL is the hardware abstraction layer which allows programmers to form device-independent code while accessing the hardware. Hence, the same code may be used in other devices with minor modifications.
- A GPIO pin within a port can be configured in several modes as input, output, analog, external interrupt, or alternate function. At the moment, we will only explain configuring the GPIO pin as input or output using STM32CubeMX.
- When the GPIO pin is configured as input, it can be left floating or its weak internal pull-up or pulldown resistor can be enabled by software.
- When the GPIO pin is configured as output, it can be configured as open-drain or push-pull mode. Its drive speed can also be selected depending on the application.

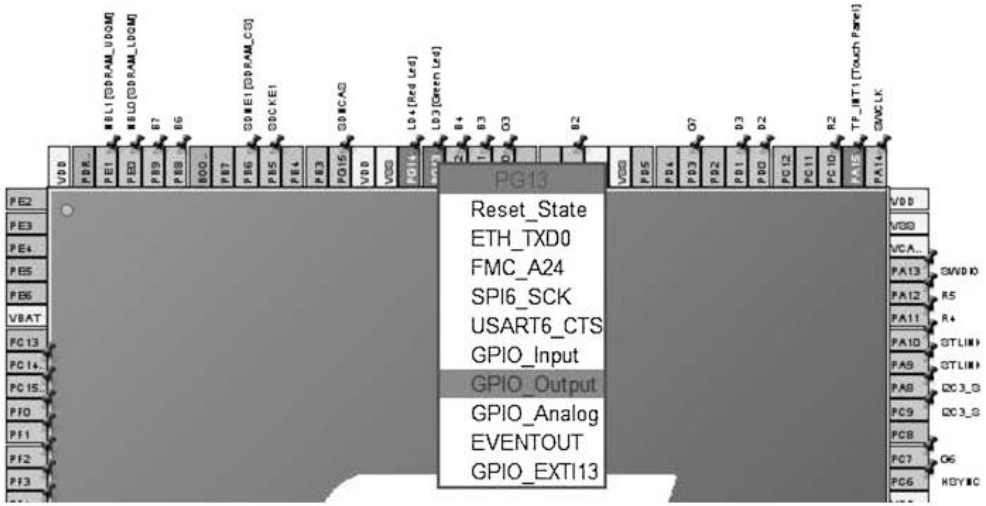


Fig. 4.10 GPIO configuration via STM32CubeMX

- A sample procedure while configuring and using a GPIO pin is as follows. We will assume that the project has already been created in STM32CubeIDE as explained in Sect. 3.1.5.
- To modify the pin usage, first open the “Device Configuration Tool” window by clicking “.ioc” file in the project explorer window. Then, click the GPIO pin on the STM32F4 microcontroller and select the GPIO type as in Fig. 4.10 on the “Device Configuration Tool” window.
- If the GPIO is selected as input, then its pull-up/pull-down resistor can be activated using “GPIO Pull-up/Pull-down” option under the “System Core—GPIO” section.
- If the mode is selected as output, then speed can be configured through “Maximum output speed” and mode can be configured through “GPIO mode” under the “System Core— GPIO” section.
- The main file will be generated when the “.ioc” file is saved with changes. Hence, STM32CubeIDE will form necessary functions to set up GPIO registers via HAL library functions

- We can use GPIO pins through HAL library functions in C language. After configuring the pin, the level of the pin can be read using the function HAL_GPIO_ReadPin() in the code if it is configured as input.
- If the GPIO pin is configured as output, then it can be set to logic level 1 or 0 by the function HAL_GPIO_WritePin() in the code. Alternatively, the function HAL_GPIO_TogglePin() can be used to toggle the output level.

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx,uint16_t
    GPIO_Pin)
/*
GPIOx: The port name where x can be (A...H) to select the
    specific port for the device.
GPIO_Pin: The pin to be read. This parameter can be one of
    GPIO_PIN_0 ...  GPIO_PIN_15
*/

void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin,
    GPIO_PinState PinState)
/*
GPIO_Pin: The pin to be written to. This parameter can be one of
    GPIO_PIN_0 ... GPIO_PIN_15
PinState: Specifies the value to be written to the selected bit.
    This parameter can be one of the GPIO_PinState value
GPIO_BIT_RESET: To clear the port pin

GPIO_BIT_SET: To set the port pin
*/

void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
/*
GPIO_Pin: The pin to be toggled. This parameter can be one of
    GPIO_PIN_0 ... GPIO_PIN_15
*/
```

- Next, we will provide example codes to show how these functions can be used to control GPIO pins.
- Our first code, given in Listing 4.1, uses the HAL library functions to toggle the green LED (connected to pin PG13) on the STM32F4 board in an infinite loop using delays. In order to observe how this code runs, create an empty project under STM32CubeIDE and configure pin PG13 as output.
- Then, use the code given in Listing 4.1 to toggle the green LED on the STM32F4 board. The code also consists of an alternative way of performing the same operation. This section is labeled as "Alternatively" within the code.

Listing 4.1 Toggle the onboard green LED by delay func

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, G
HAL_Delay(1000);

HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, G
HAL_Delay(1000);

// Alternatively
/*
HAL_GPIO_TogglePin(GPIOG,GPIO_PIN_13);
HAL_Delay(1000);
*/

/* USER CODE END WHILE */
/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- The second code, given in Listing 4.2, toggles the red and green LEDs on the STM32F4 board via user button presses. Here, pins PG13 and PG14 (connected to the green and red LEDs) are set as output. The pin PA0 (connected to the user button) is set as input. In order to observe how this code runs, create an empty project and configure the input and output pins via STM32CubeMX interface. Then, use the code given in Listing 4.2 to toggle the green and red LEDs via the user button. Within the code, there is an additional delay function to prevent switch bouncing due to button presses.

Listing 4.2 Toggle the onboard red and green LEDs by the user button presses, the

```
/* USER CODE BEGIN PV */
uint32_t counter = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 2 */
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET)
{
HAL_Delay(100); // Avoid Switch Bouncing
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_SET);
}
else
{
HAL_Delay(100); // Avoid Switch Bouncing
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
}
/* USER CODE END WHILE */
/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- The third code, given in Listing 4.3, has two counters. The first counter can be used to turn on and off the green and red LEDs on the STM32F4 board via increasing its value at each button press. When mod four of the first counter has value zero, both LEDs turn off. When mod four of the first counter value has value one, only the red LED turns on. When mod four of the first counter value has value two, only the green LED turns on. When mod four of the first counter value has value three, both LEDs turn on. The second counter increases in a continuous manner in an infinite loop.

Listing 4.3 Turn on the green and red LEDs by specific button press count values, the C code

```
/* USER CODE BEGIN PV */
uint32_t counter1 = 0, counter2 = 0;
/* USER CODE END PV */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET)
{
while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET);
counter1++;

if (counter1 % 4 == 0)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
}
else if (counter1 % 4 == 1)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_SET);
}
else if (counter1 % 4 == 2)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
}
else if (counter1 % 4 == 3)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_SET);
}
}
counter2++;
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- In Listing 4.3, there is no function added to the code to avoid switch bouncing. Instead, a hardware circuit is formed by the internal pull-down resistor and 100 ηF external capacitor (available on the STM32F4 board). Still some glitches may be observed when the button is pressed. There is a while loop placed in the code to prevent the multiple toggling operations at each button press. However, these loops block other routines, such as counter increment. The reader can observe this by placing a breakpoint to the counter increment line and pressing the button.
- The fourth code, given in Listing 4.4, is the modified form of the code given in Listing 4.3. It uses an additional shift register to eliminate switch bouncing.
- In Listing 4.4, there is a 16-bit shift register for each pin input and the read value is shifted through it. When the 0x7FFF pattern is observed for the user button, then LEDs turn on and off accordingly. The reader can observe that this code does not block the counter increment by placing a breakpoint to the counter increment line and pressing the button.

Listing 4.4 Turn on the green and red LEDs by specific button press count values with switch bouncing handled, the C code

```
/* USER CODE BEGIN PD */
#define DEBOUNCE_PATTERN 0x7FFF
/* USER CODE END PD */

/* USER CODE BEGIN PV */
uint32_t shiftRegButton;
uint32_t counter1 = 0, counter2 = 0;
/* USER CODE END PV */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
shiftRegButton = (shiftRegButton << 1) + HAL_GPIO_ReadPin(GPIOA,
    GPIO_PIN_0);
HAL_Delay(1);

if (shiftRegButton == DEBOUNCE_PATTERN)
{
counter1++;

if (counter1 % 4 == 0)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
}
else if (counter1 % 4 == 1)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_SET);
}
else if (counter1 % 4 == 2)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
}
else if (counter1 % 4 == 3)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_SET);
}
}

counter2++;
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

Listing 4.5 Toggle the onboard green LED by delay functions, the C++ code

```
#include "mbed.h"

//DigitalOut greenLED(LED1);
//DigitalOut greenLED(PG_13);

#define GRN_LED_MASK 0x2000
//#define GRN_LED_MASK 0b10000000000000
PortOut greenLED(PortG, GRN_LED_MASK);

int main()
{
while (true)
{
greenLED = 0;
//greenLED.write(0);
//greenLED = 0;

thread_sleep_for(1000);

//greenLED = 1;
//greenLED.write(1);
greenLED = GRN_LED_MASK;

thread_sleep_for(1000);

//Alternatively
/*
greenLED = !greenLED;
thread_sleep_for(1000);
*/
}
}
```

Listing 4.6 Toggle the onboard red and green LEDs by the user button presses, the C++ code

```
#include "mbed.h"

DigitalOut greenLED(LED1);

DigitalOut redLED(LED2);
//DigitalIn button(BUTTON1);

#define BUTTON_MASK 0x0001
//#define BUTTON_MASK 0b00000001
PortIn button(PortA, BUTTON_MASK);

int main()
{
greenLED = 1;
redLED = 0;
while (true)
{
if (button == 1)
{
thread_sleep_for(100);
greenLED = 0;
redLED = 1;
}
else
{
thread_sleep_for(100);
greenLED = 1;
redLED = 0;
}
}
}
```

Listing 4.7 Turn on the green and red LEDs by specific button press count values, the C++ code

```
#include "mbed.h"

DigitalOut greenLED(LED1);
DigitalOut redLED(LED2);
DigitalIn button(BUTTON1);

int counter1 = 0, counter2 = 0;

int main()
{
greenLED = 0;
redLED = 0;

while (true)
{
if (button)
{
while (button);
counter1++;

if (counter1 % 4 == 0)
{
greenLED = 0;
redLED = 0;
}
else if (counter1 % 4 == 1)

{
greenLED = 0;
redLED = 1;
}
else if (counter1 % 4 == 2)
{
greenLED = 1;
redLED = 0;
}
else if (counter1 % 4 == 3)
{
greenLED = 1;
redLED = 1;
}
}

counter2++;
}
}
```

Listing 4.8 Turn on the green and red LEDs by specific button press count values with switch bouncing handled, the C++ code

```
#include "mbed.h"
#define DEBOUNCE_PATTERN 0x7FFF

DigitalOut greenLED(LED1);
DigitalOut redLED(LED2);
DigitalIn button(BUTTON1);

int counter1 = 0, counter2 = 0;
uint32_t shiftRegButton;

int main()
{
greenLED = 0;
redLED = 0;

while (true)
{
shiftRegButton = (shiftRegButton << 1) + button;
thread_sleep_for(1);

if (shiftRegButton == DEBOUNCE_PATTERN)
{
counter1++;

if (counter1 % 4 == 0)
{
greenLED = 0;
redLED = 0;
}
else if (counter1 % 4 == 1)
{
greenLED = 0;
redLED = 1;

}
else if (counter1 % 4 == 2)
{
greenLED = 1;
redLED = 0;
}
else if (counter1 % 4 == 3)
{
greenLED = 1;
redLED = 1;
}
}

counter2++;
}
}
```

The Interrupt Concept in Embedded Systems

- The interrupt usage is almost inevitable in embedded systems. On the other hand, interrupts depend on a signal generated either by hardware or another dedicated software module. Hence, handling and using the interrupt is slightly different from standard programming practices.
- An embedded system should be interacting with the outside world by its nature. Moreover, it should respond to a request coming from the user. One option to handle such requests is forming an infinite loop in the code and checking requests in a continuous manner. Although this method is useful in its own rights, it blocks the CPU.
- To overcome this problem and freeing up the CPU from waiting in an infinite loop, a different programming approach is needed. The interrupt concept comes into play for this purpose. Within this approach, asynchronous or periodic requests can be handled without letting the CPU to wait for them indefinitely. One can think of interrupt as a response mechanism to an input signal such that when it occurs, a predefined function is executed by the CPU. The difference of the interrupt is that when the required input is not fed to the microcontroller, the CPU performs its usual synchronous operation. Hence, it does not wait for an event to occur in the infinite loop.

- To handle GPIO interrupts, we should set up the microcontroller such that related GPIO pins should be able to generate interrupt requests. In other words, the related pins should be allowed to generate interrupts. Hence, the programmer should adjust properties of each pin based on its registers.
- The user should also set priority to each interrupt request. In other words, the user should rank the interrupt requests coming from different buttons.
- As these settings are done, the pins will be ready to form interrupt requests. Besides, the CPU should be set to accept and handle these requests. This is done by setting a special register within the CPU.
- As the microcontroller is set up to handle the interrupt requests, let us observe what happens when they occur. When a button connected to the pin is pressed, the voltage difference there is sensed by an edge detector circuit connected to the pin of the microcontroller. As a result, a flag is set indicating that the pin generated an interrupt request.

- A special module handles this interrupt request. To do so, it first checks whether the request is allowed or not. Then, the module checks priority of the incoming interrupt such that it should precede the existing operation.
- What we mean by responding to the interrupt request is as follows. The CPU is diverted to the function to be executed as a response to the interrupt request. This function is called interrupt service routine (ISR) or callback function.
- As a note, these specific callback functions are defined within the code. However, they are not called by their name in the code. Instead, the hardware signal generated by the associated button press calls each function.
- As the CPU starts executing the callback function, the related flag should be reset. Hence, the CPU knows that the interrupt is handled and there is no need to pay attention to it again. As the operation within the callback function is done, the CPU turns back to its previous operation.

- The GPIO interrupt example we have provided falls into the maskable interrupts case. This means that the user can disable an interrupt request within this group. Hence, it may not be served at all.
- There is another group called non-maskable interrupts. As the name implies, an interrupt request within this group must be handled. These interrupts represent key operations such that when not served, they cause the microcontroller to malfunction.
- Reset is one such interrupt. As a matter of fact, when we debug our code and embed it on the microcontroller, a reset signal is generated such that the CPU goes to the beginning of the code to be executed. We may also generate the reset signal by pressing the reset button on the board. When this signal is generated, the CPU stops what it is doing and turns back to the beginning of the code.

Interrupts in the STM32F4 Microcontroller

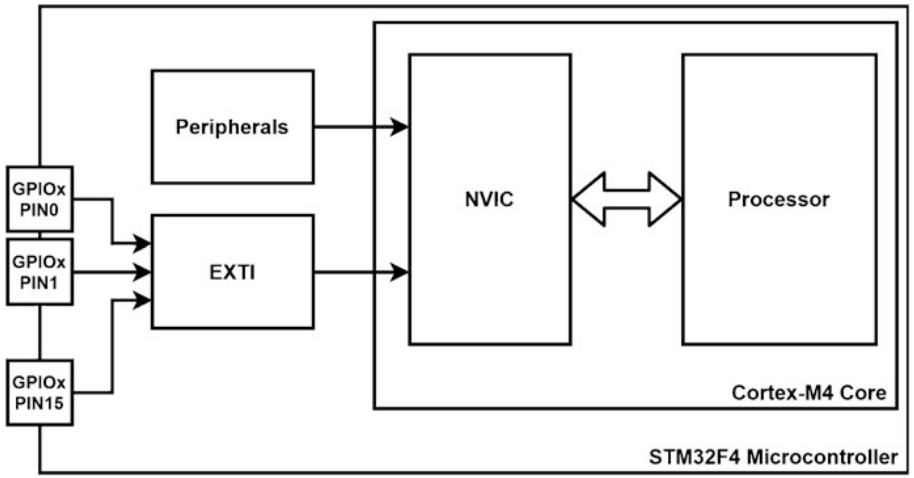


Fig. 5.1 Block diagram of the STM32F4 interrupt-based modules

- There are four main blocks related to interrupt handling in the STM32F4 microcontroller.
- The first block contains peripheral units (such as timers, ADC, and UART modules) which can generate interrupt requests.
- Within the second block, GPIO pin interrupts are handled by the extended interrupts and events controller (EXTI).
- In the third block, all interrupt requests (whether generated by a peripheral unit or EXTI) are handled by the nested vector interrupt controller (NVIC).
- In the fourth block, accepted interrupt requests are processed by the CPU. Next, we will focus on each module separately to emphasize its role in interrupt handling and execution stages.

Interrupt Operations in Peripheral Units

- As explained in the previous section, peripheral units can generate interrupt requests. This is done in several ways. For example, a GPIO pin can generate an interrupt request when it observes a low to high or high to low voltage transition.
- The timer can generate periodic interrupt requests. The ADC module can generate an interrupt request when the analog to digital conversion operation is finished.
- The UART module can generate an interrupt request after receiving or submitting a packet.
- The EXTI module handles GPIO interrupt operations in the STM32F4 microcontroller. Therefore, we will introduce this module next.
- There is a general structure for interrupt operation via peripheral units. Each peripheral unit has a specific register to enable or disable its interrupts. Besides, the unit has a flag indicating the interrupt request. In other words, if an interrupt request is generated by the peripheral unit, a flag is raised to indicate that. Then, the request is handled by the NVIC module.

Extended Interrupts and Events Controller

- The STM32F4 microcontroller has a dedicated module, called extended interrupts and events controller (EXTI), to handle external interrupts from some peripheral units.
- As can be seen in Fig. 5.1, the EXTI module stands between the GPIO pins and NVIC module. We provide a more detailed block diagram of the EXTI module in Fig. 5.2.

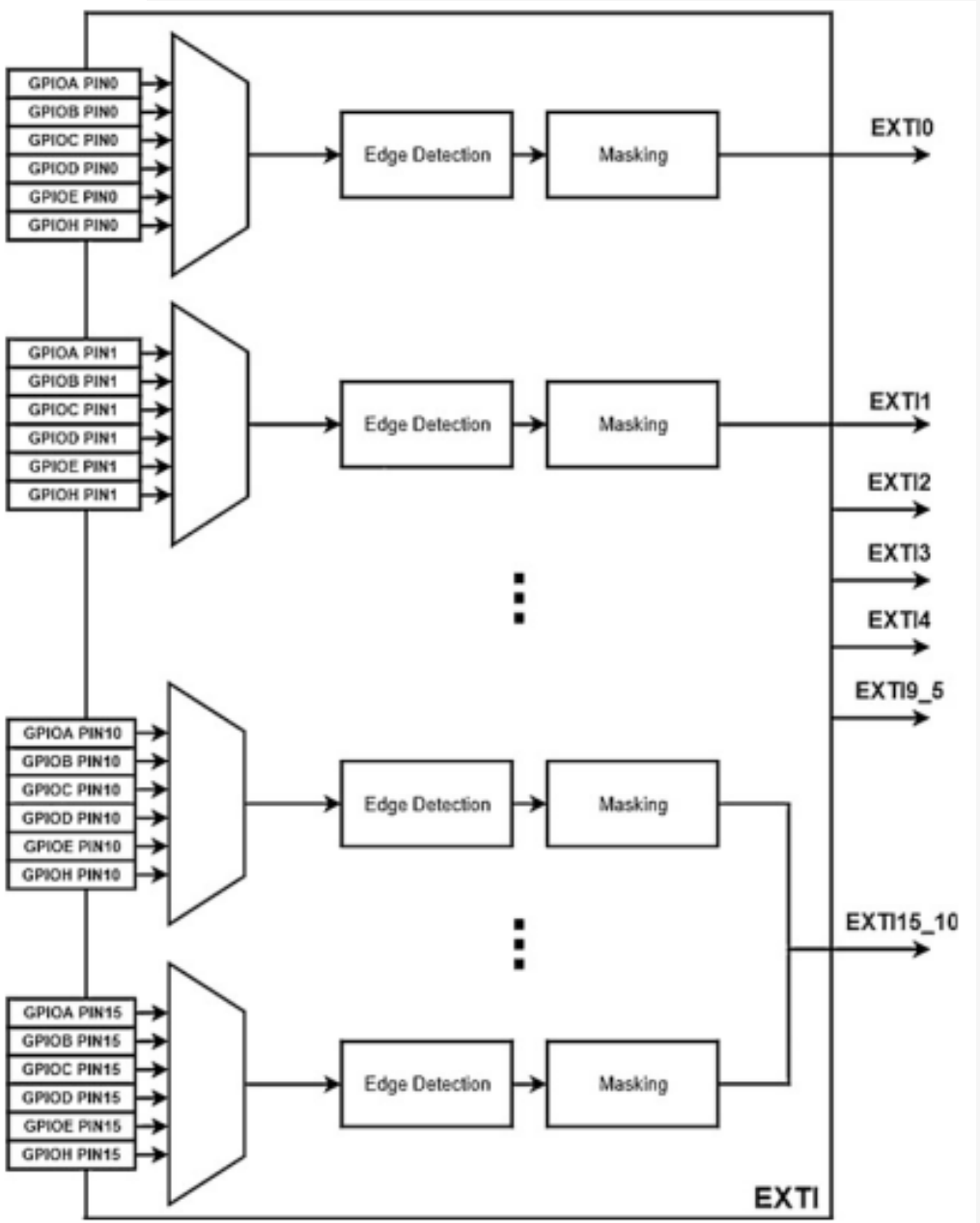


Fig. 5.2 Detailed block diagram of the EXTI module

- As can be seen in Fig. 5.2, GPIO pins are grouped within themselves and fed to separate multiplexers. Each multiplexer has an output pin called EXTI0, EXTI1, and so on.
- To be more specific, each zeroth pin of all ports is connected to the same multiplexer. Output of it is called EXTI0. Likewise, each first pin of all ports is connected to another multiplexer having output EXTI1.
- Moreover, each EXTI output is not directly connected to the NVIC module. EXTI outputs 1 to 4 are directly connected to NVIC.
- EXTI outputs 5 to 9 are grouped together and fed as a single input to NVIC. Likewise, EXTI outputs 10 to 15 are grouped together and fed as another single input to NVIC.
- Hence, the EXTI module groups GPIO pins such that they can be connected to the NVIC module. The drawback of this setup is that we cannot distinguish two interrupts originating from the same group. As an example, it is not possible to distinguish interrupts from GPIO pins PA6 and PA7.

The EXTI module can be used to enable and configure GPIO interrupts. This is done by its dedicated registers. Related to this, the module also allows masking an interrupt for its specific output.

As for configuration, the EXTI module allows generating a GPIO interrupt request by observing a low to high or high to low transition in the pin. Hence, the user can generate an interrupt by either an active high or low input from the button connected to that pin.

As the EXTI module is used to enable the GPIO interrupt for a specific pin and the required transition (either low to high or high to low) is observed there, an interrupt request is generated. Hence, a bit corresponding to the EXTI line is set. Afterward, NVIC handles the operations to be explained next.

Nested Vectored Interrupt Controller

- The nested vectored interrupt controller (NVIC) manages interrupts from peripheral units and internal exceptions originating from the CPU (such as fault handling).
- If a single interrupt request comes from a peripheral unit while the CPU is in thread mode (performing its usual sequential operation), then NVIC checks whether that request is allowed or not. It also checks whether the CPU accepts interrupt requests in general.
- If this is the case, then NVIC directs the CPU to execute the corresponding callback function for the incoming interrupt request.
- Hence, the CPU goes to the handler mode. As the callback function is executed, the CPU turns back to thread mode waiting for the next interrupt request. This is the simplest scenario for an interrupt operation.

- If more than one interrupt request comes at the same time, or an interrupt request comes while another one is being serviced, then NVIC organizes their execution based on their priority level. To do so, interrupt requests coming from different sources should be ordered.
- This is done within NVIC. Although there is a standard order for interrupt priority levels, the user can adjust them by his or her needs while forming the project. The important point here is that a priority level cannot exceed three important (and non-maskable) interrupts.
- As the priority level is set and a new interrupt request comes while another one is being serviced, the following scenarios can occur.
- First, if the CPU is in thread mode, then the priority of the incoming interrupt precedes it.
- Second, the CPU may be in handler mode responding to another interrupt with higher priority. In this case, response to the incoming low priority interrupt should wait till the end of this operation. Then, the low priority interrupt is handled.
- Third, the CPU may be in handler mode responding to an interrupt request with lower priority. Since the new incoming interrupt request has higher priority, the low priority one is suspended and the incoming new interrupt is handled. Afterward, the CPU turns back to handle the low priority interrupt request.

Interrupt Operations in the CPU

- As the NVIC module handles the interrupt request and directs the program flow, then the request is processed by the CPU.
- The first operation here is checking whether the interrupt requests are generally allowed by the CPU.
- If this condition is satisfied and CPU allows processing interrupt requests, then the corresponding callback function is executed by the CPU. Hence, CPU goes to handler mode.
- To note here, it is the user's responsibility to form the callback function. Besides, the corresponding interrupt flag should be reset within this function. Hence, the CPU will not serve it again.
- This reset operation is done automatically for most peripheral units. As the interrupt is serviced, then the CPU turns back to the thread mode again.

Interrupt Setup in the STM32F4 Microcontroller

- STM32CubeIDE adjusts all interrupt related hardware properties, such as interrupt priority and NVIC module setup, automatically as the project is compiled. Let's look at an example while configuring a GPIO pin interrupt.
- First, open the "Device Configuration Tool" window by clicking the ".ioc" file in the project explorer window. Hence, STM32CubeMX starts. Then, click on the GPIO pin on the STM32F4 microcontroller and select the GPIO type as "GPIO_EXTI" for pin interrupt source as in Fig. 5.3.

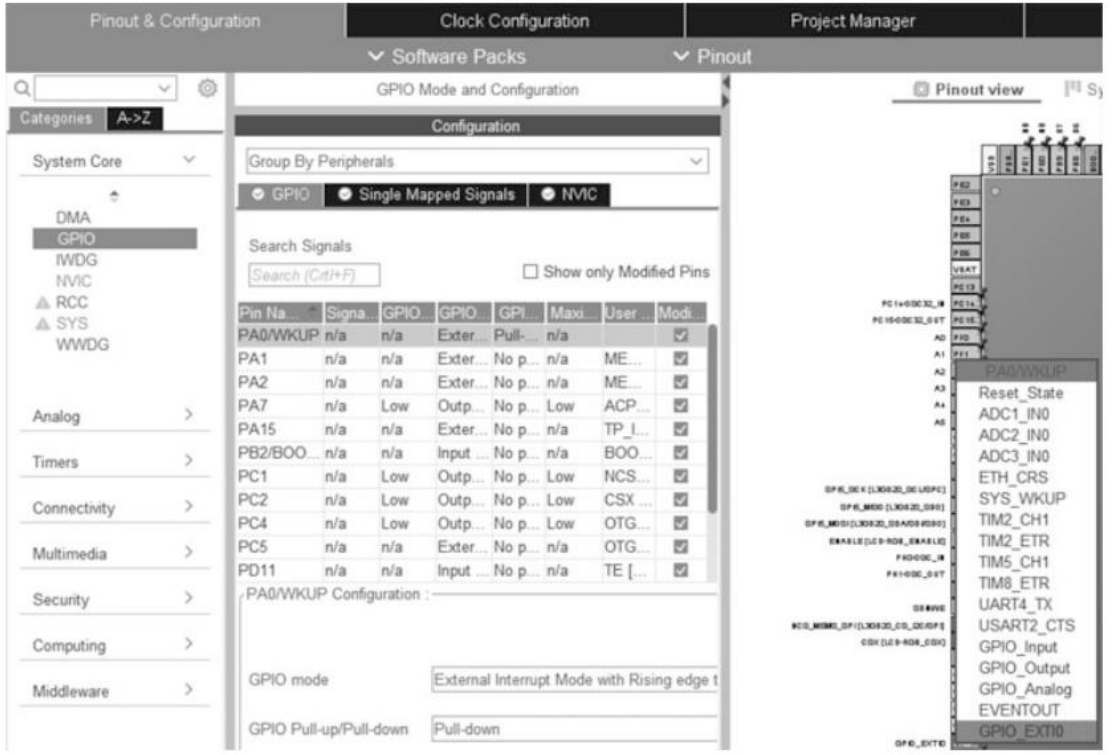


Fig. 5.3 Interrupt configuration in STM32CubeMX

- Under the “System Core—GPIO” section, GPIO mode can be selected among the following options as “external interrupt mode with rising edge trigger detection,” “external interrupt mode with falling edge trigger detection,” “external interrupt mode with rising/falling edge trigger detection,” “external event mode with rising edge trigger detection,” “external event mode with falling edge trigger detection,” and “external event mode with rising/falling edge trigger detection.”
- The difference between the event and interrupt at this stage is as follows. If an event-based option is selected, then the CPU goes to the main loop in the code after waking up from a sleep mode to be explained in Sect. 5.4.
- In the interrupt selection, the CPU starts executing the callback function after waking up from the sleep mode. Select appropriate type such as “external interrupt mode with falling edge trigger detection” here. In the same section, the pull-up/pull-down resistor can be activated using the “GPIO Pulldown” option. Select pull-up/pull-down resistor here. Afterward, enable the EXTI line interrupt under NVIC tab located in the same window or under “System Core— NVIC” section. The reader can also change the interrupt priority under this section.

Interrupt Usage via C Language

HAL library has special functions to handle interrupts. These will be of use when an interrupt-based code is written in C language under STM32CubeIDE. Next, we only provide the one related to GPIO interrupts in detail.

```
void HAL_GPIO_EXTI_Callback (uint16_t GPIO_Pin)
/*
GPIO_Pin: The pin to be connected to the corresponding EXTI line.
     This parameter can be one of GPIO_PIN_0 ... GPIO_PIN_15
*/
```

When a GPIO interrupt occurs, the NVIC module directs the CPU to its corresponding ISR. The HAL library handles the ISR such that within it the `HAL_GPIO_EXTI_Callback` is called. Without complicating the concepts, we can safely say that `HAL_GPIO_EXTI_Callback` is the callback function called when a GPIO interrupt occurs. We should define this function within the "main.c" file under STM32CubeIDE. To note here, this function is generated in weak form (without any content) automatically in the file "stm32l4xx_hal_gpio.c" when a project is generated. Hence, when the reader forgets defining the actual `HAL_GPIO_EXTI_Callback` function, the code still runs. When the user defines his or her function, then this weak function is ignored.

- The first example, with the C code given in Listing 5.2, aims to familiarize the reader with the setup and usage of interrupts.
- Here, when the onboard button (connected to PA0 pin of the STM32F4 board) is pressed and released, the green and red LEDs (connected to PG13 and PG14 pins of the STM32F4 board) toggle.
- The toggling operation is done in the callback function. As a side note, this example performs the same operation as in Listing 4.2 but with interrupt usage.

Listing 5.2 Toggle the onboard red and green LEDs by GPIO interrupt, the C cod

```
/* USER CODE BEGIN PV */
int counter1 = 0, counter2 = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)

{
if (GPIO_Pin == GPIO_PIN_0)
{
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);
}
counter2++;
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
counter1++;
/* USER CODE END WHILE */
/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

In order to execute the code in Listing 5.2, create an empty project under STM32CubeIDE and configure the LED pins of the STM32F4 microcontroller as output.

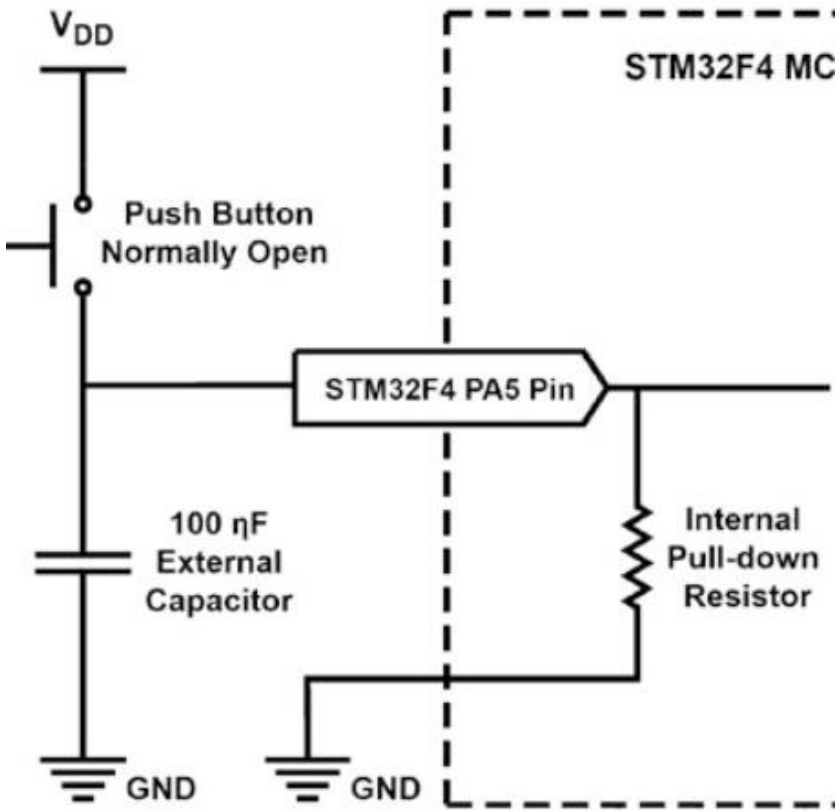
Then, configure the button pin as GPIO_EXTI and select the mode as “external interrupt mode with falling edge trigger detection.”

As the project is compiled and run, the CPU executes the callback function when the button is pressed and then released. In the callback function, the green and red LEDs toggle according to the interrupt source pin.

By adding a breakpoint, the reader can observe that the counter in the infinite loop increases while the CPU is not in handler mode.

The counter in the callback function increases only once the button is pressed and then released.

- The second example aims to familiarize the reader with the nested interrupt and priority mechanisms. To do so, we should connect an external normally open button in active high configuration and a capacitor to the pin PA5 as in Fig. 5.4. Afterward, we should configure the onboard and external button pins as GPIO_EXTI and select their mode as “external interrupt mode with falling edge trigger detection.”
- We should modify the callback function in Listing 5.2 with the new callback functions as in Listing 5.3. We should also change the interrupt priority of EXTI line 0 interrupt to 1 and EXTI line[9:5] interrupt to 2 while configuring interrupt sources. The reader can execute the new code with the modified setup to see the effect of these changes.



Listing 5.3 The effect of priority on GPIO interrupts, the C code

```
/* USER CODE BEGIN PV */
int counter1 = 0, counter2 = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)

{
if (GPIO_Pin == GPIO_PIN_0)
{
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
HAL_Delay(5000);
}
if (GPIO_Pin == GPIO_PIN_5)
{
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);
HAL_Delay(5000);
}
counter2++;
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
counter1++;
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- The second example is formed in two separate parts.
- In the first part, a low priority interrupt comes when the CPU is handling a high priority interrupt.
- Hence, the low priority interrupt is served after the high priority one is served.
- First, press the onboard button on the STM32F4 board. Release the onboard button, wait for 1 s, and press and release the external button.
- You can observe that the second press will not be recognized until four more seconds pass. In the second part, a high priority interrupt comes when the CPU is executing a low priority interrupt.
- Due to interrupt priority, the high priority interrupt is served immediately. Now, press and release the external button and wait for 1 s. Then, press and release the user button on the STM32F4 board.
- This time the second press will be recognized immediately. This example shows how the priority mechanism works.

Listing 5.4 Toggle the onboard green LED using an interrupt, the C++ code

```
#include "mbed.h"

InterruptIn userButton(BUTTON1);
DigitalOut greenLED(LED1);

void getButtons()
{
greenLED = !greenLED;
}

int main()
{
greenLED = 0;
userButton.fall(&getButtons);
__enable_irq();

while (true);
}
```

- As can be seen in Listing 5.4, the CPU waits in an infinite loop formed by the code line while(true).
- When the user presses the button, this loop is interrupted.
- The CPU executes the function ISR_callback.
- Afterward, it turns back and waits in the infinite loop for another interrupt.

Power Management in Embedded Applications

- Power management is closely related to the interrupt usage. Therefore, we will form the link between power management and interrupt usage.
- Embedded systems can be used in stand-alone applications in which they depend on battery. In order to extend working time of the overall system, we have three options.
  - First, we can select a battery with larger capacity.
  - Second, we can design a system with battery recharging capability. These two options are related to the power supply side.
  - The third option is decreasing power consumption of the embedded system.
- Improving the battery capacity and using microcontroller power management options together provide the best possible combination.



Power management in microcontrollers starts by decreasing the power consumption of the device. To do so, all peripheral units should be turned off when not used so that they do not dissipate power. This is done by inhibiting the clock signal sent to them. As a note, all peripheral units are turned off after resetting the microcontroller. It is the responsibility of the programmer to turn on the peripheral unit to be used when needed. Besides, the user should also select the appropriate power mode for the microcontroller.

The microcontroller performs its operations in active (run) mode. However, this mode has the highest power consumption. Therefore, we should let the microcontroller run in an appropriate low power mode whenever possible.

The Link Between Power Management and Interrupt Usage

- One of the most important usage areas of interrupts is letting the CPU in low power modes. This is done as follows.
- If the operation to be performed by the CPU is infrequent, one option is letting the CPU to run continuously in an infinite loop to wait for the event to be responded. Since the event is rare, most of the power will be dissipated while waiting for the event to occur.
- The next option is setting the system such that it stays in an appropriate low power or sleep mode. When the external event occurs, it triggers an interrupt. Afterward, the CPU wakes up, performs the operation related to the event in active mode, and goes back to its previous low power mode again. Hence, the system stays in active mode for a short period of time or when only needed. For the remaining times, the system stays in low power mode and does not dissipate power as it does in the active mode.

Let’s consider an actual example on the usage of interrupts to decrease the overall power consumption in operation.

In our example, we press a button to open a gate. We can safely assume that the button press is rare.

If the interrupt-based setup is not used, the system waits in active mode all the times. Meanwhile, the power consumption will be maximum.

On the other hand, if the button press generates an interrupt, then we can let the system to sleep till the interrupt occurs.

Hence, it dissipates the lowest possible power during this time.

The system will be in active mode only while executing the callback function.

This will lead to the overall power consumption to be minimal.

This scenario indicates the importance of interrupt usage in power management.

Battery as Power Supply

The embedded system is powered by battery most of the times when used in a stand-alone application. Hence, the reader should know fundamental properties of the battery such that the stand-alone application can be realized efficiently.

There are rechargeable and disposable batteries. As the name implies, the rechargeable battery can be used more than once. Although this may seem reasonable, the charging circuit increases the overall cost. Besides, rechargeable batteries are expensive.

As a final note, lifetime of a rechargeable battery is around 1000– 2000 charging. On the other hand, the disposable battery can be used without any charging circuitry. It is cheap. However, the disposable battery can only be used once and should be replaced when depleted.



- Capacity of a battery is provided in milliampere hour (mAh) by the manufacturer. This indicates the current value the battery can provide within 1 h in predefined voltage level.
- The other explanation is as follows. Capacity indicates how many hours can the battery be used when the mentioned current is drawn from it.
- Let’s consider an example to explain this concept. Assume that we have a battery with 2000 mAh capacity. Assume further that we draw 150 mA current during 15s and 2 mA current during 45s from the battery within 1 min.
- We can calculate the time needed for the battery to deplete as follows. Every minute, the average current needed will be 150 × 1/4 + 2 × 3/4 = 39 mA. Therefore, the time needed for the battery to deplete can be found as 2000 mAh / 39 mA = 51.28 h in this usage scenario.

Power Management in the STM32F4 Microcontroller

- The STM32F4 microcontroller has several key features related to power management. These can be summarized as follows.
- The microcontroller allows multiple independent power supply usage. This leads to reducing power consumption while some peripherals are supplied at higher voltages.
- The microcontroller has several power modes. These allow the programmer to benefit from the microcontroller resources while saving power.
- Multiple interrupt sources can be used to wake up the CPU from a low power mode. Hence, different pin and peripheral configurations may be applied.
- For example, the STM32F4 microcontroller can be woken up with user interaction by a button connected to a GPIO pin or periodically using timers. The microcontroller has a battery backup domain. This can be used to keep some peripheral units active (including RTC to be introduced in Chap. 6) and certain backup registers (to be introduced in Chap. 9) without the main power supply. This allows saving the last state of the microcontroller when the main power is lost.

Power Modes

- The STM32F4 microcontroller supports four power modes as run (active), sleep, stop, and standby. These are tabulated in Table 5.2.

Table 5.2 The STM32F4 microcontroller power modes

Power mode	Regulator mode	CPU/SRAM/peripherals active	Max. clock speed (MHz)
Run	Main-scale 1 (OverDrive)	All active	180
	Main-scale 1	All active	168
	Main-scale 2 (OverDrive)	All active	168
	Main-scale 2	All active	144
	Main-scale 3	All active	120
Sleep	Main-scale 1 (OverDrive)	SRAM/peripherals	180
	Main-scale 1	SRAM/peripherals	168
	Main-scale 2 (OverDrive)	SRAM/peripherals	168
	Main-scale 2	SRAM/peripherals	144
	Main-scale 3	SRAM/peripherals	120
Stop	Main-scale 3	SRAM	–
	Main-scale 3 (UnderDrive)	SRAM	–
	Low power-scale 3	SRAM	–
	Low power-scale 3 (UnderDrive)	SRAM	–
Standby	Power-down mode	–	–

- Let’s take a closer look at Table 5.2. As can be seen in this table, our first option for operation is the run mode. This mode does not have any power saving property. Hence, all peripherals and clock sources can be activated within it.
- The user has five options for setting the main CPU clock between 180 and 120 MHz in the run mode based on the regulator used.
- Our second option is the sleep mode. The CPU is stopped in this mode. However, SRAM and all peripherals are active. These can wake up the CPU by an interrupt.
- Our third option is the stop mode. This mode is called deep sleep mode in Arm® literature. The stop mode achieves the lowest power consumption while retaining the contents of SRAM and registers. The CPU can be woken up from the stop mode by any of the EXTI line (the EXTI line source can be one of the 16 external lines, the PVD output, the RTC alarm/wake-up/tamper/time stamp events, the USB OTG FS/HS wakeup, or the Ethernet wakeup).
- Our fourth option is the standby mode. Only the RTC, 4 kB of backup SRAM, and 20 backup registers (80 bytes) are active in this mode. The CPU can exit from the standby mode either by an external reset pin, a watchdog reset, a rising edge on one of the enabled WKUPx pins or when a RTC event occurs. After waking up from standby mode, code execution restarts in the same way as after a reset operation.

STM32CubeMX for Power Usage Analysis

- STM32CubeMX has a power consumption calculator (PCC) tool which can be used to estimate active, inactive, and average current consumption values for a given scenario. The PCC tool also estimates the battery life when the one to be used in operation is selected. The PCC tool can be used by setting up multiple steps such that different power modes, clock source and frequency, and peripheral units can be selected at each step. Hence, the user can check complex usage scenarios with the tool.



Fig. 5.5 The PCC tool under STM32CubeMX

Listing 5.2 Toggle the onboard red and green LEDs by GPIO interrupt, the C cod

```
/* USER CODE BEGIN PV */
int counter1 = 0, counter2 = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
if (GPIO_Pin == GPIO_PIN_0)
{
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);
}
counter2++;
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
counter1++;
/* USER CODE END WHILE */
/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- On the left side menu, a *VDD* value and battery can be selected. On the right-hand side of the view, a new step can be added by clicking the “New Step” button. While adding or editing a step, power and memory settings, clocks, peripheral units, and other additional settings, such as duration of a step, can be adjusted. When a new step is added and there is a transition error between power modes, then the step turns to red on the list. Results are shown as graphs in the display section of the tool.



- We next provide an example on the usage of the PCC tool. Within this example, we will calculate the current consumption when the CPU runs for 10 s when the code in Listing 5.2 is executed.
- To do so, open the ".ioc" file in the project explorer window, click on "Tools," tab and add a new step. Here, choose the power mode as "RUN," power scale as "Scale-3-Low," memory fetch type as "Flash," and CPU frequency as "25 MHz." Enter the step duration as 10 s or 10000 ms and click the "Enable IPs from Pinout" button. You may also enable IPs (peripherals) from the peripheral selection menu in the middle.
- Once the configuration is complete, click the "Add" button. You can observe the current consumption profile graph. As can be seen there, approximately 11.41 mA average current is consumed in operation. Now, edit the added step and change the step duration as 0.1 s or 100 ms. Add another step, choose power mode as "STOP_NM (Normal Mode)" and clock configuration as "Regulator_LP," and set the step duration as 9.9 s or 9900 ms. This time, the average current consumption drops down to approximately 401.2 µA. We will use the PCC tool in the following sections to emphasize the usage of power modes in general.

Power Modes in C Language

Sleep Mode

- HAL library functions related to the sleep mode are as follows. The function HAL_PWR_EnterSLEEPMode can be used to enter the sleep mode. The function HAL_PWR_EnableSleepOnExit enables sleep-on-exit when returning from handler to thread mode. Likewise, the function HAL_PWR_DisableSleepOnExit disables sleep-on-exit when returning from the handler to thread mode.

```
void HAL_PWR_EnterSLEEPMode (uint32_t Regulator, uint8_t
    SLEEPEntry)
/*
Regulator: Specifies the regulator state. This parameter is not
    used for the STM32F4 family. Hence, we should set it as
    PWR_MAINREGULATOR_ON.
SLEEPEntry: Specifies if the sleep mode is entered with WFI or
    WFE instruction. This parameter can be one of
    PWR_SLEEPENTRY_WFI or PWR_SLEEPENTRY_WFE
*/

void HAL_PWR_EnableSleepOnExit (void)

void HAL_PWR_DisableSleepOnExit (void)
```

- We next provide an example on the usage of sleep mode in C language. To do so, we repeat the example given in Listing 5.2 to toggle onboard LEDs using GPIO interrupts. However, instead of actively waiting in an infinite loop when the button is not pressed, this time we let the CPU wait in sleep mode. To do so, the reader should create an STM32CubeIDE project first. Then, the infinite loop in Listing 5.2 should be replaced by the code block in Listing 5.7. As the project is executed, the reader can observe that both counters in the infinite loop and callback function do not increase when the button is not pressed.

Listing 5.7 A sample code for the usage of sleep mode functions, in C language

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
HAL_SuspendTick();
HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
HAL_ResumeTick();

counter1++;
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- The reader can observe that there are two extra functions used in Listing 5.7 as *HAL_SuspendTick()* and *HAL_ResumeTick().*
- These are related to the **SysTick** and clock setup to be explained in detail in Chap. 6. However, we had to cover them here since these two modules work in sleep mode.
- SysTick automatically generates interrupts at every millisecond and wakes up the CPU. To avoid this, we had to disable it before the sleep operation and enable it afterward as in Listing 5.7.
- We can analyze the power consumption of the modified project by the PCC tool. Here, the GPIOA and GPIOG ports are enabled. The power scale is set to “Scale-3- Low,” memory fetch type to “Flash,” and CPU frequency to “25 MHz” for both run and sleep modes. Under these conditions, the average current consumption will be 11.1 mA and 3.1 mA for the run and sleep modes, respectively.

```
void HAL_PWR_EnterSTOPMode (uint32_t Regulator, uint8_t
    STOPEntry)
 /*
Regulator: Specifies the regulator state. This parameter is not
    used for the STM32F4 family. Hence, we should set it as
    PWR_MAINREGULATOR_ON.
 STOPEntry: Specifies if stop mode is entered with WFI or WFE
    instruction. This parameter can be one of PWR_STOPENTRY_WFI
    or PWR_STOPENTRY_WFE.
 */

 HAL_StatusTypeDef HAL_PWREx_EnterUnderDriveSTOPMode (uint32_t
    Regulator, uint8_t STOPEntry)
```

Stop Mode

- HAL library functions related to the stop mode are as follows. The function HAL_PWR_EnterSTOPMode can be used to enter the stop mode 0 or 1 by specifying the regulator state. Likewise, the function HAL_PWREx_EnterUnder DriveSTOPMode leads the microcontroller to the underdrive mode with low power consumption.

Listing 5.8 A sample code for the usage of stop mode functions, in C language

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
HAL_SuspendTick();
HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI
    );
HAL_ResumeTick();

counter1++;
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- We will next consider usage of the stop mode by repeating the example in Listing 5.2. This time we let the CPU wait in stop mode instead of actively waiting in an infinite loop when the user button is not pressed. As in the previous example, the reader should first create a project. Then, he or she should use the code block in Listing 5.8 to replace the infinite loop in Listing 5.2 and run the project. The reader can observe that the counter in the infinite loop does not increase when the button is not pressed.
- To note here, the functions HAL_SuspendTick() and HAL_ResumeTick() are used to stop and start the SysTick, respectively. They are added to avoid starting the SysTick during the debug session. They may not be needed if the project is compiled and run without debugging it.

```
void HAL_PWR_EnterSTANDBYMode (void)

void HAL_PWR_EnableWakeUpPin (uint32_t WakeUpPinx)
/*
WakeUpPinx: Specifies which Wake-Up pin to enable. This
    parameter can only be PWR_WAKEUP_PIN1.
*/

__HAL_PWR_CLEAR_FLAG (_FLAG_)
/*
__FLAG__: specifies the flag to clear. This parameter can be one
     of the PWR_FLAG_WU or PWR_FLAG_SB
*/
```

Standby Mode

- HAL library functions related to the standby mode are as follows. The function HAL_PWR_EnterSTANDBYMode can be used to enter the standby mode. The function HAL_PWR_EnableWakeUpPin can be used to enable the wake-up PINx functionality. The macro __HAL_PWR_CLEAR_FLAG should be used to clear the flag if it will be used in the wake-up process for the standby mode. We provide detailed information on these functions next.

- We next provide an example on the usage of standby mode. Here we wake up the CPU with the onboard button connected to PA0, toggle the onboard LEDs connected to pins PG13 and PG14, and then put CPU back to standby mode. To do so, the reader should first create an empty project and configure the LED pins of the STM32F4 microcontroller as output. Then, he or she should configure the button pin as SYS_WKUP and select the mode as “external interrupt mode with falling edge trigger detection.” Then, the reader should use the code block in Listing 5.9 and run the project.

- As the project is executed, the green and red LEDs turn on one after the other with 1 s delay. Then, they turn off together. However, we do not turn off the LEDs in code. Instead, they turn off since the GPIO ports connected to them are closed. As the CPU turns on after the standby mode, it resets itself. Hence, the LEDs turn on and off again.

Listing 5.9 A sample code for the usage of standby mode functions, in

```
/* USER CODE BEGIN 2 */
HAL_GPIO_TogglePin(GPIOG,GPIO_PIN_13);
HAL_Delay(1000);
HAL_GPIO_TogglePin(GPIOG,GPIO_PIN_14);
HAL_Delay(1000);

__HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);
HAL_PWR_EnableWakeUpPin(PWR_WAKEUP_PIN1);
HAL_PWR_EnterSTANDBYMode();
/* USER CODE END 2 */
```

Lab4 solution

```
#include <stdint.h>

int *memloc;

int main(void)
{
    memloc = (int *)0x20000040;
    unsigned char temp = 0xF0;


    for(int i=0; i<4; i++){
        *memloc = 0x00000000;
        for(int j=0;j<4;j++){
                *memloc = *memloc<<8;
                *memloc += temp;
                temp += 0x01;
        }
        memloc++;
    }

    memloc = (int *)0x20000040;

    for(int i=0; i<4; i++){
        for(int j=0;j<4;j++){
                if(*memloc & 0x01){
                        *memloc = *memloc>>8;
                        *memloc &= 0x00FFFFFF;
                }
                else{
                        *memloc = *memloc>>8;
                        *memloc |= 0xFF000000;
                }
        }
        memloc++;
       }

        for(;;);
}
```